

HDOC Day-19

Implementation of SGPA and CGPA Calculation Using Nested Loops & Decision-Making Constructs in C

1. Problem Statement & Identification

- **Problem:** Write a C program to calculate the Semester Grade Point Average (SGPA) for m semesters and overall Cumulative Grade Point Average (CGPA) with strict input validation for global_id, m, n, code, credit, and grade.
- **Input:** global_id, m, n, code, credit, grade.
- **Output:** Semester-wise SGPA, total cumulative credits (cumulative_TC), and final CGPA.

2. Algorithm

1. **Start**
2. Define the variables and their types: global_id, m, n, semester, course, code, credit, GP, semester_TC, semester_TGC, cumulative_TC, and cumulative_TGC as integers; grade as a character; and SGPA and CGPA as floating-point variables.
3. Get global_id.

4. Validate global_id: If global_id is not positive, display an appropriate error message and request it again. Repeat until a positive value is entered.
5. Get m.
6. Validate m: If m is not positive, display an appropriate error message and request it again. Repeat until a positive value is entered.
7. Set the cumulative values: cumulative_TC = 0, cumulative_TGC = 0.
8. Repeat Steps 8.1 to 8.8 for m semesters:
 - **8.1** Set the semester totals: semester_TC = 0, semester_TGC = 0.
 - **8.2** Get n.
 - **8.3** Validate n: If n is not positive, display an appropriate error message and request it again. Repeat until a positive value is entered.
 - **8.4** Repeat Steps 8.4.1 to 8.4.9 for n courses:
 - **8.4.1** Get code.
 - **8.4.2** Validate code: If it is not positive, display an error message and request it again. Repeat until a positive value is entered.
 - **8.4.3** Get credit.

- **8.4.4** Validate credit: If it is not positive, display an error message and request it again. Repeat until a positive value is entered.
- **8.4.5** Get grade.
- **8.4.6** Validate grade: Accept only O, A, B, C, D, E, or P. If the grade is invalid, display an error message and request it again. Repeat until a valid grade is entered.
- **8.4.7** Find GP corresponding to grade using the university's conversion scale: O = 10, A = 9, B = 8, C = 7, D = 6, E = 5, P = 4.
- **8.4.8** Calculate the semester total grade credits: $\text{semester_TGC} = \text{semester_TGC} + (\text{GP} * \text{credit})$.
- **8.4.9** Calculate the semester total credits: $\text{semester_TC} = \text{semester_TC} + \text{credit}$.
- **8.5** Calculate the SGPA of the current semester: $\text{SGPA} = \text{semester_TGC} / \text{semester_TC}$.
- **8.6** Display semester and SGPA.
- **8.7** Add the semester grade credits to the cumulative grade credits: $\text{cumulative_TGC} = \text{cumulative_TGC} + \text{semester_TGC}$.

- **8.8** Add the semester credits to the cumulative credits: $\text{cumulative_TC} = \text{cumulative_TC} + \text{semester_TC}$.

9. Calculate the CGPA: $\text{CGPA} = \text{cumulative_TGC} / \text{cumulative_TC}$.

10. Display global_id, m, cumulative_TC, and CGPA.

11. **Stop.**

3. Test Case Table

Test Case Type	Global ID	m	n	Course Inputs (Code, Credit, Grade)	Expected Output	Actual Output	Status
Valid Input	101	2	Sem 1: 2	S1: (101, 4, O), (102	Sem 1 SGPA = 9.57	Sem 1 SGPA = 9.57	Pass

Test Case Type	Global ID	m	n	Course Inputs (Code, Credit, Grade)	Expected Output	Actual Output	Status
			Sem 2: 1	, 3, A) S2: (201, 3, B)	Sem 2 SGPA = 8.00 Cumulative TC = 10, CGPA = 9.10	Sem 2 SGPA = 8.00 Cumulative TC = 10, CGPA = 9.10	
Invalid Input	-5 → 102	0 → 1	-1 → 1	(101, -2 → 4,	Prompt Error Message	Prompt Error Message	Pass

Test Case Type	Global ID	m	n	Course Inputs (Code, Credit, Grade)	Expected Output	Actual Output	Status
s (Validation Check)				'Z' → 'A')	ges & Re-enter SGPA = 9.00, CGPA = 9.00	ges & Re-enter SGPA = 9.00, CGPA = 9.00	
Boundary Input	1	1	1	(101, 1, P)	Sem 1 SGPA = 4.00	Sem 1 SGPA = 4.00	Pass

Test Case Type	Global ID	m	n	Course Inputs (Code, Credit, Grade)	Expected Output	Actual Output	Status
					Cumulative TC = 1, CGPA = 4.00	Cumulative TC = 1, CGPA = 4.00	

4. C Source Code

```
#include <stdio.h>

int main(void) {
    int global_id, m, n, semester, course, code, credit, GP;
    int semester_TC, semester_TGC, cumulative_TC, cumulative_TGC;
    char grade;
    float SGPA, CGPA;

    do {
        printf("Enter Global ID: ");
        scanf("%d", &global_id);
        if (global_id <= 0) {
            printf("Error: Global ID must be positive. Please try again.\n");

```

```

    }
} while (global_id <= 0);

do {
    printf("Enter number of semesters (m): ");
    scanf("%d", &m);
    if (m <= 0) {
        printf("Error: Number of semesters must be positive. Please try
again.\n");
    }
} while (m <= 0);

cumulative_TC = 0;
cumulative_TGC = 0;

for (semester = 1; semester <= m; semester++) {
    printf("\n===== SEMESTER %d =====\n",
semester);

    semester_TC = 0;
    semester_TGC = 0;

    do {
        printf("Enter number of courses for Semester %d (n): ", semester);
        scanf("%d", &n);
        if (n <= 0) {
            printf("Error: Number of courses must be positive. Please try
again.\n");
        }
    } while (n <= 0);

    for (course = 1; course <= n; course++) {
        printf("\n--- Course %d ---\n", course);

        do {
            printf("Enter Course Code: ");
            scanf("%d", &code);
            if (code <= 0) {
                printf("Error: Course code must be positive. Please try
again.\n");
            }
        } while (code <= 0);
    }
}

```



```

    }
} while (code <= 0);

do {
    printf("Enter Course Credit: ");
    scanf("%d", &credit);
    if (credit <= 0) {
        printf("Error: Course credit must be positive. Please try
again.\n");
    }
} while (credit <= 0);

int valid_grade = 0;
do {
    printf("Enter Grade (O, A, B, C, D, E, P): ");
    scanf(" %c", &grade);

    switch (grade) {
        case 'O': case 'o': GP = 10; valid_grade = 1; break;
        case 'A': case 'a': GP = 9; valid_grade = 1; break;
        case 'B': case 'b': GP = 8; valid_grade = 1; break;
        case 'C': case 'c': GP = 7; valid_grade = 1; break;
        case 'D': case 'd': GP = 6; valid_grade = 1; break;
        case 'E': case 'e': GP = 5; valid_grade = 1; break;
        case 'P': case 'p': GP = 4; valid_grade = 1; break;
        default:
            printf("Error: Invalid grade! Only O, A, B, C, D, E, P are
accepted. Try again.\n");
            valid_grade = 0;
            break;
    }
} while (!valid_grade);

semester_TGC = semester_TGC + (GP * credit);
semester_TC = semester_TC + credit;
}

SGPA = (float)semester_TGC / semester_TC;
printf("\n---> Semester %d SGPA: %.2f\n", semester, SGPA);

```

```

        cumulative_TGC = cumulative_TGC + semester_TGC;
        cumulative_TC = cumulative_TC + semester_TC;
    }

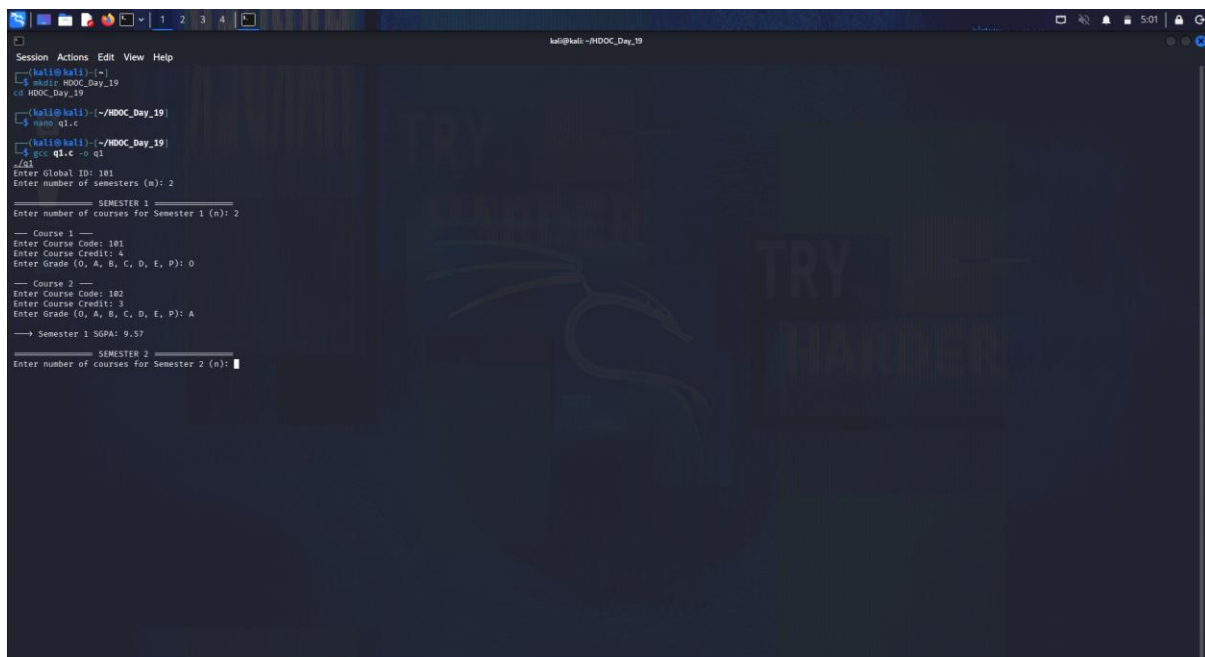
    CGPA = (float)cumulative_TGC / cumulative_TC;

    printf("\n===== FINAL RESULTS =====\n");
    printf("Global ID      : %d\n", global_id);
    printf("Semesters Completed : %d\n", m);
    printf("Total Credits (TC) : %d\n", cumulative_TC);
    printf("CGPA          : %.2f\n", CGPA);
    printf("===== \n");

    return 0;
}

```

5. Compilation, Execution & Screenshots



```

kali@kali:~/HDOOC_Day_19
$ gcc q1.c
$ ./q1
Enter Global ID: 101
Enter number of semesters (n): 2
===== SEMESTER 1 =====
Enter number of courses for Semester 1 (n): 2
----- Course 1 -----
Enter Course Code: 101
Enter Course Credit: 4
Enter Grade (O, A, B, C, D, E, P): O
----- Course 2 -----
Enter Course Code: 102
Enter Course Credit: 3
Enter Grade (O, A, B, C, D, E, P): A
----> Semester 1 SGPA: 9.57
===== SEMESTER 2 =====
Enter number of courses for Semester 2 (n):

```